Mark Verma

Bengaluru, India • markvrma@gmail.com • +91 93021 53281 • github.com/markvrma • markvrma.github.io

Experience

314e Corp — *Bengaluru, India*
Software Development Engineer — *Jan 2025 – Present*
Associate Software Development Engineer — *Jul 2024 – Jan 2025*

- Clinical document product needed reliable extraction across 26 entity types; owned the ML stack and fine-tuned a 27B vision-language model to **96 percent test accuracy in production**
- Entity extraction and the agentic feedback loop spanned several teams' needs; led development of both the entity extraction flow and the agentic feedback flow, working with cross-functional teams to align requirements, interfaces and rollout so the two flows shipped as one connected system, while mentoring 2 junior software engineers
- Every model-family change forced a training rewrite; wrote a 7,600-line fine-tuning package (SkyPilot, Temporal, ClearML) with one dispatch layer, so **Gemma 3, Qwen3.5-9B and Qwen3.6-27B train through the same code**
- Retraining was manual and models could silently get worse; built a reviewer-feedback loop (stratified datasets, automated deploy) behind a per-entity F1 gate, so **no model regressing beyond a tolerance scaled to its prior score ships**
- F1 drops could mean a worse model or a change in reviewers' labelling convention, which need opposite fixes; built a weekly drift monitor where an LLM agent reads page OCR through a read-only MCP server, so **regression is separated from labelling change by checking the document itself**
- Four LLM providers and three inference engines produced inconsistent outputs; wrote a schema-driven extraction layer with per-field token-logprob confidence, **giving all of them one validated output contract**
- LLM self-confidence was a poor approval signal; trained a calibrated XGBoost model on logprobs, OCR grounding and embeddings over 99,427 reviews, **halving false approvals (1.4% vs 2.7%)**, **cutting calibration error 0.139 to 0.004 and retiring 83% of manual entity review**
- Serving was slow to scale and slow to start; moved Transformers to vLLM to SGLang on RunPod serverless and baked weights and pre-warmed kernels into the image, reaching **64 concurrent jobs per worker with no cold-start cost**
- Training took 90 hours and exhausted 80 GB of host RAM because a cap bounded sample count, not bytes; switched to map-style lazy loading with VRAM-sized batches, **cutting training from 90 to 25 hours**

Banach Technologies — *Singapore (Remote)*
Machine Learning Intern — *Apr 2024 – Jul 2024*

- Built and deployed ML models against live market APIs to execute hedging strategies, cutting data-processing latency through targeted performance tuning and covering the core trading libraries with unit tests

Alemeno — *Maharashtra (Remote)*
Software and ML Intern — *Jul 2023 – Apr 2024*

- Built a preprocessing pipeline for semantic-segmentation models over large-scale GIS raster data, implementing distributed computer-vision algorithms including Douglas-Peucker and Jarvis March, and deployed Django services on AWS behind end-to-end CI/CD

Bewgle — *Bengaluru (Remote)*
Data Analyst Intern — *May 2022 – Jul 2022*

- Applied NLP to proprietary Amazon review datasets for product-trend insight, automating preprocessing in Python and Bash to cut data redundancy 90%

Projects

brigd — *Rust, LangChain-style agent orchestration, Claude Code, ratatui* — github.com/markvrma/brigd

- Built a Rust agent orchestrator based on LangChain's principles of composable chains, planner-driven agents and schema-validated structured output, which plans a task into a staged chain of parallel Claude agents: a planner session emits a schema-constrained JSON flow (model, effort, permissions, files per agent), each stage runs its agents concurrently, and stage $N+1$ starts only once every agent in stage N has written its result, re-reading the flow before each stage so agents can rewrite the plan mid-run
- Ran every agent as its own PTY session in an isolated git worktree with hook-reported status, validated flows to reject two editing agents in one workspace, persisted thread state to disk so any run resumes, and shipped the planner as an installable skill
- Wrote the 4,400-line Rust TUI (42 tests): live terminal panes, split panes and tabs, per-agent diff trees from session transcripts, and pause, terminate and revert controls

msm — *Rust* — github.com/markvrma/msm

- Ported a Python terminal music player to a 7,300-line Rust binary (75 tests) streaming YouTube Music and a local library through mpv over its JSON IPC socket, with Last.fm scrobbling, pixelated 256-colour album art and an audio visualizer, shipped with CI, cargo-dist releases and a Homebrew tap

rumpy — *Rust* — github.com/markvrma/rumpy

- Built a zero-dependency ndarray library on a flat-buffer, shape and strides layout, with stride-0 broadcasting, NaN-propagating axis reductions and a cache-friendly ikj matmul, 32 tests and a median-of-reps harness benchmarked against NumPy

Open Source

- Authored 8 pull requests across PyMC, MDAnalysis and sktime with 4 merged, including generalizing the PyMC Multinomial moment computation to arbitrary batch dimensions in pymc-devs/pymc#5476 and removing 91 lines of dead code; MDAnalysis work completed under Google Season of Docs
- Merged 19 of 21 pull requests to a collaborative Discord bot, adding a vote-to-skip queue and Billboard Hot 100 integration and fixing a recurring API-timeout defect, placing 2nd in an open-source competition (github.com/markvrma/discord-bot-sample)

Publication

Trained a CNN to 98.4% accuracy for medical-waste classification and deployed it as an interactive TensorFlow.js web application, published as first author in IEEE IATMSI 2022 and cited 25 times (ieeexplore.ieee.org/document/10119431)

Skills

Languages: Python, C++, Rust, SQL, Bash
ML and LLM: PyTorch, Hugging Face Transformers, vLLM, SGLang, XGBoost, LoRA fine-tuning, vision-language models, structured outputs, probability calibration, inference optimization, model evaluation, drift detection, SHAP, RAG, agentic tool use, multi-agent orchestration, Model Context Protocol, NumPy, Pandas, scikit-learn
Infrastructure: Docker, Kubernetes, Ray Serve, Temporal, SkyPilot, ClearML, MLflow, RunPod, FastAPI, gRPC, PostgreSQL, AWS, Git, Linux

Education

IIITM Gwalior — B.Tech, Computer Science, SGPA 8.67 — Gwalior, India • Graduated 2024